

1. Finite Precision Arithmetic

1.1. Introduction

In computing, numbers are represented using a finite number of bits, which imposes limitations on both the range and precision of values that can be stored. These limitations lead to certain phenomena such as overflow, underflow, and precision loss. This document covers basic concepts of numerical representation including storage formats (endianness), integer numbers, floating-point numbers, and related errors.

1.2. Storage Formats: Endianness

When storing multi-byte numbers, the order in which the bytes are stored depends on the system's architecture. There are two main types of byte ordering:

- **Big Endian:** The most significant byte (MSB) is stored at the smallest memory address. For example, the 4-byte hexadecimal number `0x12345678` is stored as:

Address:	0	1	2	3
Value:	12	34	56	78

- **Little Endian:** The least significant byte (LSB) is stored at the smallest memory address. The same 4-byte number `0x12345678` would be stored as:

Address:	0	1	2	3
Value:	78	56	34	12

Endianness does not affect how numbers are interpreted, but it can be important when reading or writing binary data to files or communicating across different systems.

1.3. Integer Numbers

Integers are represented in computers as fixed-length binary numbers.

Binary Representation of an Integer

In binary representation, an integer number is represented as a sequence of bits (binary digits), where each bit corresponds to a power of 2. The general form for an n -bit binary

1. Finite Precision Arithmetic

number is:

$$N = b_{n-1} \cdot 2^{n-1} + b_{n-2} \cdot 2^{n-2} + \dots + b_1 \cdot 2^1 + b_0 \cdot 2^0 \quad (1.1)$$

where each b_i is a binary digit (either 0 or 1), and n is the total number of bits.

Example: Binary Representation of 13

Consider the decimal number 13. Its binary representation using 4 bits is:

$$13_{10} = 1101_2 \quad (1.2)$$

This can be expanded as:

$$13 = 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \quad (1.3)$$

$$8 + 4 + 0 + 1 = 13 \quad (1.4)$$

2^3	2^2	2^1	2^0	
1	1	0	1	=13
8	4	2	1	

In this example, the binary number 1101_2 represents the decimal number 13.

Example: Binary Addition with Overflow

Let's consider adding two 4-bit binary numbers that result in an overflow.

$$7_{10} = 0111_2 \quad \text{and} \quad 11_{10} = 1011_2 \quad (1.5)$$

The sum of these two binary numbers is $0111_2 + 1011_2$, which should give 18 in decimal, but with only 4 bits available, an overflow occurs.

$$0111_2 + 1011_2 = 10010_2 \quad (\text{overflow occurs as the result requires 5 bits}) \quad (1.6)$$

2^3	2^2	2^1	2^0	
0	1	1	1	
				+
1	0	1	1	
1	0	0	1	0
				Overflow

Here, the binary addition results in a 5-bit number 10010_2 , but since we're using only 4 bits, the leftmost bit (the overflow bit) is discarded, leading to an incorrect result. The correct result, 18 in decimal, cannot be represented with just 4 bits, and thus overflow occurs.

1.3.1. Signed vs. Unsigned Integers

- **Unsigned:** All bits are used to represent the magnitude of the number, so an n -bit unsigned integer can represent values from 0 to $2^n - 1$.
- **Signed (Two's Complement):** The most significant bit is used as the sign bit. An n -bit signed integer can represent values from -2^{n-1} to $2^{n-1} - 1$.

1.3.2. Overflow Example

Consider an 8-bit unsigned integer, which can store values between 0 and 255. If we add two numbers, 200 and 100, the result should be 300. However, since 300 exceeds the maximum value of 255, it results in an overflow:

$$200 + 100 = 300 \equiv 44 \pmod{256} \quad (1.7)$$

In this case, the computed result is 44 due to overflow.

1.4. Floating-Point Numbers

Floating-point numbers provide an approximation of real numbers. They are typically represented in IEEE 754 format, which consists of three parts: the sign bit, the exponent, and the mantissa (or significand).

Here is a table with the characteristic numbers for the different types

Table 1.1.: Comparison of Half, Single, Double, and Quad Precision Floating-Point Numbers

Precision	Total Bits	Mant. Bits	Exp. Bits	Smallest Non-Zero Value	Smallest Value	Largest Value
Half	16	10	5	5.96×10^{-8}	6.10×10^{-5}	6.55×10^4
Single	32	23	8	1.40×10^{-45}	1.18×10^{-38}	3.40×10^{38}
Double	64	52	11	4.94×10^{-324}	2.22×10^{-308}	1.80×10^{308}
Quad	128	112	15	6.47×10^{-4966}	3.36×10^{-4932}	1.19×10^{4932}

1.4.1. Single Precision

Single-precision floating-point numbers use 32 bits:

1. Finite Precision Arithmetic

- 1 bit for the sign
- 8 bits for the exponent
- 23 bits for the mantissa

The general form of a floating-point number is:

$$(-1)^s \times (1.m) \times 2^e \quad (1.8)$$

where s is the sign bit, m is the mantissa, and e is the exponent.

1.4.2. Precision Limitations and Cancellation Error

The precision of floating-point numbers is limited by the number of bits used for the mantissa. For single precision, this limits the precision to approximately 7 decimal digits. One common source of error is *cancellation*, which occurs when subtracting two nearly equal numbers, causing significant digits to be lost.

Consider two single-precision floating-point numbers:

$$a = 1.0000001, \quad b = 1.0000000 \quad (1.9)$$

Subtracting these gives:

$$a - b = 1.0000001 - 1.0000000 = 0.0000001 \quad (1.10)$$

However, due to the limited precision of floating-point numbers, the result might be stored as zero if the difference is smaller than the precision threshold.

1.4.3. Overflow and Underflow in Floating-Point Numbers

- **Overflow:** Occurs when a floating-point number exceeds the maximum representable value. For example, in single precision, the largest representable number is approximately 3.4×10^{38} . Any number larger than this results in overflow.
- **Underflow:** Occurs when a number is too small to be represented, closer to zero than the smallest representable positive value (approximately 1.2×10^{-38} in single precision). Such a number is rounded down to zero.

code example for an overflow:

```
#include <stdio.h>

int main() {
    unsigned char a = 200;
    unsigned char b = 100;
    unsigned char result = a + b;
    printf("Result: \u00d\n", result); // Output will be 44 due to overflow
    return 0;
}
```

1.5. Mathematical Accuracy, Convergence, and Numerical Stability

When solving scientific problems numerically, it is important to distinguish between two different questions:

1. Is the *mathematical method* correct/converging?
2. Is the *numerical implementation* reliable on a finite-precision computer?

These questions are related, but they are not the same.

Mathematical Accuracy

Mathematical accuracy describes how close an approximation is to the exact solution. If x^* is the exact value and x_n an approximation, then the error is

$$e_n = x_n - x^*. \quad (1.11)$$

A method is called accurate if this error becomes small. For example, polygon approximations of a circle become more accurate when the number of sides increases.

Convergence

A sequence of approximations x_n is called convergent if

$$\lim_{n \rightarrow \infty} x_n = x^*. \quad (1.12)$$

This means that the approximation approaches the exact mathematical result when the resolution, number of steps, or iteration count increases.

Examples:

- Increasing the number of polygon sides to approximate π
- Using smaller time steps in differential equations
- Using finer grids in numerical integration

Convergence is therefore a property of the mathematical algorithm.

Numerical Stability

Numerical stability describes how rounding errors, truncation errors, or finite machine precision affect the computation. Even if a method is mathematically convergent, the implementation may fail because computers store numbers only approximately.

Typical sources of instability are:

- subtraction of nearly equal numbers (cancellation)

1. Finite Precision Arithmetic

- division by very small numbers
- accumulation of rounding errors over many iterations
- overflow or underflow

A numerically stable algorithm controls such errors so that they do not grow significantly.

Why These Concepts Are Different

A method can be mathematically correct but numerically poor, for example *The formula may converge in exact arithmetic, but fail in floating-point arithmetic.* Likewise, two algebraically identical formulas can have very different numerical behavior. This is one of the central lessons of computational physics.

1.6. Example: Archimede's computation of π

One of the oldest numerical methods to approximate π goes back to Archimedes. The idea is simple:

- Use a regular polygon *inside* a circle (inscribed polygon, P_{in}).
- Use a regular polygon *outside* a circle (circumscribed polygon, P_{out}).

The perimeter of the inner polygon is smaller than the circumference of the circle, while the perimeter of the outer polygon is larger. For a unit circle ($r = 1$), the circumference is

$$C = 2\pi \tag{1.13}$$

and therefore

$$P_n^{\text{in}} < 2\pi < P_n^{\text{out}}. \tag{1.14}$$

Dividing by 2 gives lower and upper bounds for π . As the number of sides n increases, both polygon perimeters converge to the circle circumference.

1.6.1. Basic Approximation Formula

Let

- s_n = side length of the inscribed regular n -gon
- t_n = side length of the circumscribed regular n -gon

Then

$$\pi_n^{\text{in}} = \frac{n}{2}s_n, \quad \pi_n^{\text{out}} = \frac{n}{2}t_n. \tag{1.15}$$

and

$$\pi_n^{\text{in}} < \pi < \pi_n^{\text{out}}. \tag{1.16}$$

1.6.2. Starting Values

For a hexagon ($n = 6$) in a unit circle we find

$$s_6 = 1, \quad t_6 = \frac{2}{\sqrt{3}}. \quad (1.17)$$

These values are exact.

1.6.3. Doubling the Number of Sides

The key idea of Archimedes is to repeatedly double the number of sides:

$$6 \rightarrow 12 \rightarrow 24 \rightarrow 48 \rightarrow 96 \rightarrow \dots \quad (1.18)$$

This requires formulas for computing s_{2n} and t_{2n} from s_n and t_n .

1.6.4. Inscribed Polygon

Let us derive an expression for the inscribed polygon:

For a unit circle, the side length of the inscribed regular n -gon is

$$s_n = 2 \sin\left(\frac{\pi}{n}\right). \quad (1.19)$$

We want to derive a recursion for the doubled polygon, that is, for s_{2n} .

Full derivation

Starting from the definition,

$$s_{2n} = 2 \sin\left(\frac{\pi}{2n}\right). \quad (1.20)$$

Now use the half-angle identity

$$\sin\left(\frac{x}{2}\right) = \sqrt{\frac{1 - \cos x}{2}}. \quad (1.21)$$

With $x = \frac{\pi}{n}$, this becomes

$$\sin\left(\frac{\pi}{2n}\right) = \sqrt{\frac{1 - \cos(\pi/n)}{2}}. \quad (1.22)$$

Therefore,

$$s_{2n} = 2 \sqrt{\frac{1 - \cos(\pi/n)}{2}} = \sqrt{2 - 2 \cos\left(\frac{\pi}{n}\right)}. \quad (1.23)$$

Next, express $\cos(\pi/n)$ in terms of s_n . From

$$s_n = 2 \sin\left(\frac{\pi}{n}\right), \quad (1.24)$$

1. Finite Precision Arithmetic

we get

$$\sin\left(\frac{\pi}{n}\right) = \frac{s_n}{2}. \quad (1.25)$$

Using the identity

$$\cos^2 x = 1 - \sin^2 x, \quad (1.26)$$

we obtain

$$\cos\left(\frac{\pi}{n}\right) = \sqrt{1 - \sin^2\left(\frac{\pi}{n}\right)} = \sqrt{1 - \left(\frac{s_n}{2}\right)^2}. \quad (1.27)$$

Since for the polygon construction we have $0 < \pi/n < \pi/2$, the cosine is positive, so we take the positive square root. Thus,

$$\cos\left(\frac{\pi}{n}\right) = \sqrt{1 - \frac{s_n^2}{4}} = \frac{1}{2}\sqrt{4 - s_n^2}. \quad (1.28)$$

Insert this into the expression for s_{2n} :

$$s_{2n} = \sqrt{2 - 2\left(\frac{1}{2}\sqrt{4 - s_n^2}\right)} = \sqrt{2 - \sqrt{4 - s_n^2}}. \quad (1.29)$$

Hence the recursion relation is

$$s_{2n} = \sqrt{2 - \sqrt{4 - s_n^2}}. \quad (1.30)$$

Compact derivation

$$s_{2n} = 2 \sin\left(\frac{\pi}{2n}\right) \quad (1.31)$$

$$= 2 \sqrt{\frac{1 - \cos(\pi/n)}{2}} \quad (1.32)$$

$$= \sqrt{2 - 2 \cos(\pi/n)} \quad (1.33)$$

$$= \sqrt{2 - 2 \sqrt{1 - \sin^2(\pi/n)}} \quad (1.34)$$

$$= \sqrt{2 - 2 \sqrt{1 - (s_n/2)^2}} \quad (1.35)$$

$$= \sqrt{2 - \sqrt{4 - s_n^2}}. \quad (1.36)$$

Numerical remark

For large n , the side length s_n becomes small, so $\sqrt{4 - s_n^2}$ is very close to 2. Therefore the subtraction

$$2 - \sqrt{4 - s_n^2} \quad (1.37)$$

suffers from cancellation and can lose numerical accuracy. This is why the equivalent stable reformulation is preferable in floating-point arithmetic.

Stable Formula

We can simply transform the equation to a stable one

$$s_{2n}^2 = 2 - \sqrt{4 - s_n^2} \quad (1.38)$$

$$= \frac{(2 + \sqrt{4 - s_n^2})(2 - \sqrt{4 - s_n^2})}{(2 + \sqrt{4 - s_n^2}) \cdot 1} \quad (1.39)$$

$$= \frac{s_n^2}{(2 + \sqrt{4 - s_n^2})}. \quad (1.40)$$

This gives the numerically better equivalent form

$$s_{2n} = \frac{s_n}{\sqrt{2 + \sqrt{4 - s_n^2}}} \quad (1.41)$$

This avoids subtracting nearly equal numbers.

1.6.5. Circumscribed Polygon

Unstable Formula

$$t_{2n} = \frac{2(\sqrt{4 + t_n^2} - 2)}{t_n} \quad (1.42)$$

For small t_n :

$$\sqrt{4 + t_n^2} \approx 2, \quad (1.43)$$

so again the subtraction

$$\sqrt{4 + t_n^2} - 2 \quad (1.44)$$

causes cancellation errors.

Stable Formula

The equivalent stable form is

$$t_{2n} = \frac{2t_n}{2 + \sqrt{4 + t_n^2}} \quad (1.45)$$

This avoids the dangerous subtraction.

1. Finite Precision Arithmetic

1.6.6. Algorithm

Starting from $n = 6$:

1. Compute s_n and t_n

2. Estimate:

$$\pi_n^{\text{in}} = \frac{n}{2}s_n, \quad \pi_n^{\text{out}} = \frac{n}{2}t_n \quad (1.46)$$

3. Update:

$$n \leftarrow 2n \quad (1.47)$$

using the recursion formulas

4. Repeat

Errors

In Fig. 1.1 shows the estimate (top) and the error (bottom) for both inscribed and circumscribed polygons with the stable and unstable formulation.

1.6. Example: Archimede's computation of π

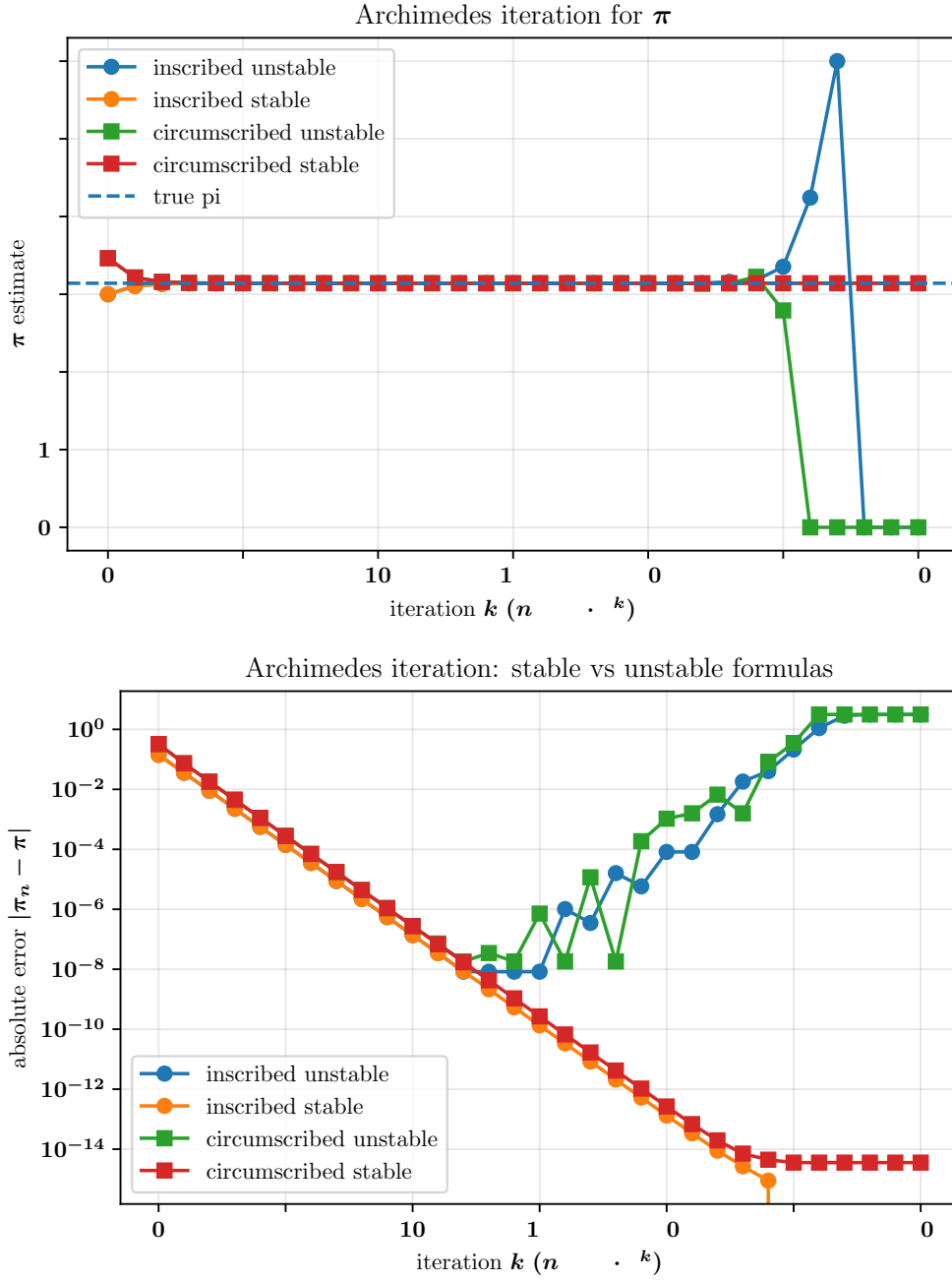


Figure 1.1.: Estimate for π (top) and error (bottom) for inscribed and circumscribed polygons for both the stable and unstable method as a function of the iteration.

